



Regression Trees vs KNN Regression in R: Non-Linear Models Explained

Tree-Based Regression

While linear regression provides a strong baseline, tree-based models take a different approach — they partition the feature space into regions, making model decisions interpretable and visual. Building on the foundation established with `reg_lm`, this section covers two tree-based approaches in `daltoolbox`: Decision Trees, which produce explicit rules, and Random Forest, which combines multiple trees for greater predictive robustness.

Decision Tree for Regression

A decision tree recursively partitions the feature space using **binary splits** on input features. In the regression setting, each leaf node predicts the **mean of the training samples** that fall into that region. Because it is **non-parametric**, the tree assumes no functional form between inputs and output — making it naturally capable of capturing **non-linearities** while remaining fully interpretable: every prediction can be traced back through a sequence of human-readable rules.

How It Works

At each node, the algorithm selects the feature and threshold that minimizes the MSE of the resulting split. This continues recursively until a stopping criterion is met.

Prediction for a new observation = mean of target values in the leaf it falls into.

Key Hyperparameter

Tree depth (`max_depth`) governs the bias-variance trade-off:

- **Shallow tree** → high bias, underfitting
- **Deep tree** → low bias, high variance, overfitting

The gap between train (R^2 : 0.859) and test (R^2 : 0.512) metrics illustrates this directly.

Strengths & Limitations

Strengths:

- Fully interpretable decision rules
- Captures non-linearities naturally
- No feature scaling required

Limitations:

- Prone to overfitting (high variance)
- Unstable — small data changes alter the tree

❗ Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and Regression Trees*. Wadsworth. — The foundational reference for CART, the algorithm underlying most decision tree implementations.

reg_dtree: Decision Tree for Regression

Setup & Data

```
# Regression Decision Tree

#install.packages("daltoolbox")
library(daltoolbox)
library(MASS)

data(Boston)
Boston <- as.matrix(Boston)

set.seed(1)
sr <- sample_random()
sr <- train_test(sr, Boston)
boston_train <- sr$train
boston_test <- sr$test
```

Training & Evaluation

```
# Training
model <- reg_dtree("medv")
model <- fit(model, boston_train)

# Evaluation
train_pred <- predict(model, boston_train)
train_eval <- evaluate(model,
  boston_train[, "medv"], train_pred)
print(train_eval$metrics)
# mse=12.68, smape=0.13, R2=0.86

test_pred <- predict(model, boston_test)
test_eval <- evaluate(model,
  boston_test[, "medv"], test_pred)
print(test_eval$metrics)
# mse=29.38, smape=0.16, R2=0.51
```

Step-by-Step

- Load `daltoolbox` and the `Boston` dataset (MASS)
- Convert to matrix with `as.matrix()` for better performance
- Split into train/test sets using `sample_random()` and `train_test()`
- Define the model with `reg_dtree("medv")` and fit it using `fit()`
- Evaluate with `evaluate()`: metrics include MSE, SMAPE, and R^2
- Compare train ($R^2=0.86$) vs. test ($R^2=0.51$) to detect overfitting — a key signal as we progress to more complex models

📌 **Key Concept:** The decision tree partitions the feature space and estimates the target variable by averaging leaf node values. It is interpretable and captures non-linearities. The main shift from classification lies in the **evaluation step**, not in the fitting logic — a pattern that will carry through the entire regression workflow.

Neighborhood-Based Regression

Building on the decision tree baseline, we now explore neighborhood-based methods for predictive regression using the `daltoolbox` package in R. KNN (K-Nearest Neighbors) estimates a target variable by computing the **average of the k nearest neighbors** in feature space — a simple, non-parametric approach that effectively captures local patterns without assuming a global model structure.

K-Nearest Neighbors for Regression

KNN is a **non-parametric, instance-based** learning method. Rather than fitting an explicit model, it stores the entire training set and makes predictions by locating the **k most similar observations** (neighbors) in feature space, then averaging their target values. Because no model is learned during training, KNN is often called a *lazy learner*. Performance is highly sensitive to **feature scaling** and the **choice of k**, both of which directly affect how distances are computed and how the bias-variance trade-off is balanced.

How It Works

For a new observation, compute the distance (e.g., Euclidean) to every training point. Select the **k nearest** neighbors and predict the **mean of their target values**.

The entire training dataset is stored and queried at prediction time — this is the essence of *lazy learning*. No compression or abstraction of the data occurs during the training phase.

Key Hyperparameter: k

k governs the bias-variance trade-off:

- **Small k** → low bias, high variance (risk of overfitting)
- **Large k** → high bias, low variance (risk of underfitting)

Feature scaling is critical — variables with larger magnitudes dominate distance computations, so standardization or normalization must be applied before fitting.

Strengths & Limitations

Strengths:

- Conceptually simple with no training phase
- Naturally captures **local, non-linear patterns**
- Adapts to complex decision boundaries

Limitations:

- Slow at prediction time — scales with dataset size
- Sensitive to irrelevant or redundant features
- No explicit model available for interpretation

❗ Altman, N. S. (1992). An introduction to kernel and nearest-neighbor nonparametric regression. *The American Statistician*, 46(3), 175–185. — The foundational reference for understanding KNN in the regression context and its statistical properties.

reg_knn: K-Nearest Neighbors for Regression

Setup & Data

```
# Regression KNN

library(daltoolbox)
library(MASS)

data(Boston)
Boston <- as.matrix(Boston)

set.seed(1)
sr <- sample_random()
sr <- train_test(sr, Boston)
boston_train <- sr$train
boston_test <- sr$test

# Training — define k
model <- reg_knn("medv", k=5)
model <- fit(model, boston_train)
```

Training & Evaluation

```
# Training evaluation
train_pred <- predict(model, boston_train)
train_eval <- evaluate(model,
  boston_train[, "medv"], train_pred)
print(train_eval$metrics)
# mse=27.03, smape=0.16, R2=0.70

# Test evaluation
test_pred <- predict(model, boston_test)
test_eval <- evaluate(model,
  boston_test[, "medv"], test_pred)
print(test_eval$metrics)
# mse=26.95, smape=0.18, R2=0.55
```

Step-by-Step

- Same data preparation as previous examples (Boston dataset, random split)
- Define the model with `reg_knn("medv", k=5)` — the main parameter is `k`
- The model predicts the mean of the `k` nearest neighbors' target variable values
- Evaluate training and test sets using the same regression metrics
- Notice that training ($R^2=0.70$) and test ($R^2=0.55$) are closer together than with the decision tree — a sign of more stable generalization

📌 **Key Concept:** KNN borrows information from neighboring observations. The parameter `k` controls the bias-variance trade-off: small `k` = high variance; large `k` = high bias. The **Experiment Line remains identical** to that of the decision tree — only the model changes.

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

